

Chapter 2

1- An Overview of *Java*

Objectives

- At the end of this chapter students should be able to:
 - Understand Primitive and Reference data types in java
 - Understand and describe literals and operators in java
 - Understand and define Statements in java
 - Define and create Objects and Classes
 - Understand and define methods in java
 - Develop and execute Java program
 - Define and manipulate arrays in java
 - Understand garbage collector
 - Know wrapper classes

Java Basic Syntax

- Java syntax is defined in the Java language specification, and the Java library is defined in the Java API.
- The Java language specification is a technical definition of the Java programming language's syntax and semantics.
- Java is case sensitive.
- Name of the program file should exactly match the public class name.
- Every statement in java should end with a semicolon.
- Follow Java naming conventions.

Java naming conventions

Name	Convention
class name	should start with uppercase letter and be a noun e.g. String, Color, Button, System, Thread etc.
Interface name	should start with uppercase letter and be an adjective e.g. Runnable, Remote, ActionListener etc.
method name	should start with lowercase letter and be a verb e.g. actionPerformed(), main(), print(), println() etc.
variable name	should start with lowercase letter e.g. firstName, orderNumber etc.
package name	should be in lowercase letter e.g. java, lang, sql, util etc.
constants name	should be in uppercase letter. e.g. RED, YELLOW, MAX_PRIORITY etc.

Java Identifiers

- Names used for classes, interfaces, variables and methods are called identifiers.
- In Java, there are several points to remember about identifiers:
 - ✓ All identifiers should begin with a letter (A to Z or a to z), an underscore (_) or **currency character (\$)** (but not **recommended**).
 - ✓ After the first character identifiers can have any combination of characters.
 - ✓ A key word cannot be used as an identifier.
 - ✓ Most importantly identifiers are case sensitive.
 - Examples of legal identifiers: age, **\$salary**, _value, _1_value
 - Examples of illegal identifiers: 123abc, -salary

Data Types in Java

➤ There are two data types available in Java:

1. Primitive Data Types
2. Reference/Object Data Types
 - Class reference type
 - Interface reference type
 - Array reference type

Primitive Data Types

- Primitive data types are predefined by the language and named by a keyword.
- There are eight primitive data types supported by Java:
 - 1) byte--- -8 -bit signed
 - 2) Short ---16- bit signed
 - 3) char-----16- bit unsigned
 - 4) int ----- -32 -bit signed
 - 5) long -----64 -bit
 - 6) float
 - 7) double
 - 8) boolean

Numerical Primitive Data Types

Name	Range	Storage Size
byte	-2^7 to $2^7 - 1$	8-bit signed
short	-2^{15} to $2^{15} - 1$	16-bit signed
int	-2^{31} to $2^{31} - 1$	32-bit signed
long	-2^{63} to $2^{63} - 1$	64-bit signed
float	Negative range: $-3.4028235\text{E}+38$ to $-1.4\text{E}-45$ Positive range: $1.4\text{E}-45$ to $3.4028235\text{E}+38$	32-bit IEEE 754
double	Negative range: $-1.7976931348623157\text{E}+308$ to $-4.9\text{E}-324$ Positive range: $4.9\text{E}-324$ to $1.7976931348623157\text{E}+308$	64-bit IEEE 754

Literals

- A *literal* is a constant value that appears directly in the program /source code.
 - ✓ Numeric literals
 - Integer literal
 - Long literal
 - Floating point literal
 - Double literal
 - ✓ Boolean literals
 - ✓ Character literals
 - ✓ String literals

Integer Literals

- By default an integer literal is treated as an int type value.
- To denote an integer literal of the long type, append it with the letter L or l.

Example

int i = 34; // 34 is an integer literal

long x = 1000000; // 1000000 is a long literal

Floating-Point Literals

- By default, a floating-point literal is treated as a double type value. For example, 5.0 is considered a double value, not a float value.
- You can make a number a float by appending the letter f or F. For example, you can use 100.2f or 100.2F for a float number.
- The double type values are more accurate than the float type values.
- Floating-point literals can also be specified in scientific notation, for example, 1.23456e+2, same as 1.23456e2, is equivalent to 123.456, and 1.23456e-2 is equivalent to 0.0123456.
- E (or e) represents an exponent and it can be either in lowercase or uppercase.

Numeric Type Conversion

- Consider the following statements:

`byte i = 100;`

`long k = i * 3 + 4;`

`double d = i * 3.1 + k / 2;`

- When performing a binary operation involving two operands of different types, Java automatically converts(promotes) the operand based on the following rules.

Conversion Rules

Type	Valid promotions
double	None
float	double
long	float or double
int	long, float or double
char	int, long, float or double
short	int, long, float or double (but not char)
byte	short, int, long, float or double (but not char)
boolean	None (boolean values are not considered to be numbers in Java)

Range increases
byte, short, int, long, float, double



Conversion...

➤ **Implicit conversion (promotion):**

`double d = 3;` (type widening)

➤ **Explicit conversion (type casting):**

`int i = (int)3.0;` (type narrowing)

`int i = (int)3.9;` (Fraction part is truncated)

What is wrong if ? `int x = 5 / 2.0;`

Casting in an Augmented Expression

➤ In Java, an augmented expression of the form **x1 op= x2** is implemented as **x1 = (T)(x1 op x2)**, where **T** is the type for **x1**.

➤ Therefore, the following code is correct.

```
int sum = 0;
```

```
sum += 4.5; // sum becomes 4 after this statement
```

sum += 4.5 is equivalent to **sum = (int)(sum + 4.5)**.

Boolean, Character and String Literals

- true and false are boolean literals.
- A character literal is a single character or an escape sequence enclosed in single quotes, for example, 'b'.
 - Escape sequences are used to indicate special characters or actions.
- A string literal consists of zero or more characters (including the escape sequences) enclosed in double quotes. Examples of string literals are:

null

"Hello World" "two\n lines"

"\"This is in quotes\""

Operators

- **Operators** are special symbols that are used to represent simple computations like addition and multiplication.
- Java provides a rich set of operators to manipulate variables.
- We can divide all the Java operators into the following groups:

The Arithmetic Operators: (assume A=10 & B=20)

Operator	Description	Example
+	Addition	A + B will give 30
-	Subtraction	A - B will give -10
*	Multiplication	A * B will give 200
/	Division	B / A will give 2
%	Modulus - Divides left hand operand by right hand operand and returns remainder	B % A will give 0

Relational Operators: used to check conditions

Operator	Description	Example
<code>==</code>	Checks if the value of two operands is equal or not, if yes then condition becomes true.	<code>(A == B)</code> is not true.
<code>!=</code>	Checks if the value of two operands is equal or not, if values are not equal then condition becomes true.	<code>(A != B)</code> is true.
<code>></code>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	<code>(A > B)</code> is not true.
<code><</code>	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	<code>(A < B)</code> is true.
<code>>=</code>	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	<code>(A >= B)</code> is not true.
<code><=</code>	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	<code>(A <= B)</code> is true.

Java Logical Operators

- Java's logical operators enable you to form more complex conditions by combining simple conditions.

1. Conditional Operators:

- Conditional AND (&&)
- Conditional OR (||)

2. Boolean logical Operators:

- Boolean logical AND (&)
- Boolean logical inclusive OR (|)
- Boolean logical exclusive OR (^)
- Logical NOT (!)

- ✓ Note: The &, | and ^ operators are also bitwise operators when they're applied to integral operands.

Conditional Operators

Conditional AND (&&)

- ✓ false && false: false
- ✓ false && true: false
- ✓ true && false: false
- ✓ true && true: true

Conditional OR (||)

- ✓ false || false: false
- ✓ false || true: true
- ✓ true || false: true
- ✓ true || true: true

Note: The conditional operators are short circuit operators since in some cases as the left side expression is evaluated we can tell that the whole expression is true or not so that the right side expressions are not evaluated.

Boolean Logical Operators

Boolean logical AND (&)

- ✓ false & false: false
- ✓ false & true: false
- ✓ true & false: false
- ✓ true & true: true

Boolean logical inclusive OR (|)

- ✓ false | false: false
- ✓ false | true: true
- ✓ true | false: true
- ✓ true | true: true

Logical NOT (!)

- ✓ !false : true
- ✓ !true: false

Boolean logical exclusive OR (^)

- ✓ false ^ false: false
- ✓ false ^ true: true
- ✓ true ^ false: true
- ✓ true ^ true: false

Assignment Operator

- After a variable is declared, you can assign a value to it by using an assignment statement.
- In Java, the equal sign (=) is used as the assignment operator.
- For example, consider the following code:

```
int y = 1; // Assign 1 to variable y  
double radius = 1.0; // Assign 1.0 to variable radius  
int x = 5* (3/ 2); // Assign the value of the expression to x  
x = y + 1; // Assign the addition of y and 1 to x  
double area = radius * radius * 3.14159; // Compute area and  
assign it to area
```

Augmented Assignment Operators

<i>Operator</i>	<i>Name</i>	<i>Example</i>	<i>Equivalent</i>
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>

Increment and Decrement Operators

<i>Operator</i>	<i>Name</i>	<i>Description</i>	<i>Example (assume i = 1)</i>
++var	preincrement	Increment var by 1 , and use the new var value in the statement	int j = ++i; // j is 2, i is 2
var++	postincrement	Increment var by 1 , but use the original var value in the statement	int j = i++; // j is 1, i is 2
--var	predecrement	Decrement var by 1 , and use the new var value in the statement	int j = --i; // j is 0, i is 0
var--	postdecrement	Decrement var by 1 , and use the original var value in the statement	int j = i--; // j is 1, i is 0

Increment and Decrement Operators (cont'd...)

`int i = 10;`
`int newNum = 10 * i++;` Same effect as `int newNum = 10 * i;`
`i = i + 1;`

`int i = 10;`
`int newNum = 10 * (++i);` Same effect as `i = i + 1;`
`int newNum = 10 * i;`

Note: Increment or decrement operators can only be applied to a **variable**—not a constant expression.

Operators for Strings

- In general you cannot perform mathematical operations on Strings, even if the strings look like numbers.
- The following are illegal (if we know that fred has type String):

`fred - 1` `"Hello"/123` `fred * "Hello"`

- For Strings, the `+` operator represents **concatenation**, which means joining up the two operands by linking them end-to-end. So `"Hello, "` + `"world."` yields the string `"Hello, world."`

Reference Data Types

- Reference variables are created using defined constructors of the classes.
- They are used to access objects.
- These variables are declared to be of a specific type that cannot be changed.

For example, Employee, Animal, String, etc.

- Class objects, interfaces and various types of array variables come under reference data types.
- Default value of any reference variable is null.
- A reference variable can be used to refer to any object of the declared type or any compatible type.

Example: **Animal animal = new Animal("giraffe");**

Java Variables

- A variable is a named location that stores a **value**.
- In order to store a value, you have to create a variable.
- Each variable has a type that determines what kind of values it can store.
- A variable must be declared and initialized before it can be used.
- The variable declaration tells the compiler to allocate appropriate memory space for the variable based on its data type.

Declaring Variables

➤ The syntax for declaring variables is:

[access modifier] datatype variableName;

```
int x;           // Declare x to be an
                  // integer variable;

double radius;  // Declare radius to
                  // be a double variable;

char a;          // Declare a to be a
                  // character variable;
```

Assigning Values to Variables

- Now that we have created some variables, we would like to store values in them.
- We do that with an **assignment statement**.

Examples:

```
x = 1;           // Assign 1 to x
```

```
radius = 1.0; // Assign 1.0 to radius
```

```
a = 'A';        // Assign A to a;
```

Declaring and Initializing in One Step

```
int x = 1;
```

```
double d = 1.4;
```

```
String name = "John";
```

```
boolean b = true;
```

```
char c = 'X';
```


Named Constants

- A named constant is an identifier that represents a permanent value.
- A constant must be declared and initialized in the same statement.
- The word **final** is a Java keyword used for declaring a constant.
- Here is the syntax for declaring a constant:

[access modifier] final datatype CONSTANTNAME = value;

Example: final double PI = 3.14159;

final int SIZE = 3;

Expressions

- An *expression* is a construct made up of variables, operators, and method invocations.
- An expression usually evaluates to a single value.
- Expressions do not represent storage locations, only values.

Examples:

`minute+1` = hour; // illegal expression

hour = `minute + 1`; // legal expression

Expressions...

➤ Java expressions are evaluated in the same way as arithmetic expressions.

For example

$$\frac{3 + 4x}{5} - \frac{10(y - 5)(a + b + c)}{x} + 9\left(\frac{4}{x} + \frac{9 + x}{y}\right)$$

is translated to an equivalent java expression:

$$(3+4*x)/5 - 10*(y-5)*(a+b+c)/x + 9*(4/x + (9+x)/y)$$

Simple Java Program

- **Java program** can be defined as a collection of objects that can communicate via invoking each other's methods.
- A Java application is a computer program that executes when you use the java command to launch the Java Virtual Machine (JVM).
- Every Java program must have at least one class.
- Every class we define usually begins with the public keyword.

Simple Java Program...

- A public class must be placed in a file that has the same name as the class plus the **.java** extension; otherwise, a compilation error occurs.
- The syntax to define a class in java is:

```
public class ClassName  
{  
//Class members  
}
```

Anatomy of a Java Program

- ➡ Class name
- ➡ Main method
- ➡ Statements
- ➡ Statement terminator
- ➡ Reserved words
- ➡ Comments
- ➡ Blocks

Cont'd...

- Let's begin with a simple Java program that displays the message **Welcome to Java!** on the console.

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Class Name

➤ In this example, the class name is Welcome.

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```


Declaring a **main()** Method

- Java program execution starts from the main() method which is a mandatory part of every Java program.
- In java main method must be defined as below; otherwise, the Java Virtual Machine (JVM) will not execute the application.

```
public static void main (String[] args){  
    // main method body  
}
```

Statement

- A statement represents an action or a sequence of actions.
- The statement `System.out.println("Welcome to Java!")` in this program is a statement to display the greeting "Welcome to Java!".

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Statement Terminator

- Every statement in Java ends with a semicolon (;).

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```



Statement Terminator

Reserved words

- Reserved words or keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program.
- For example, when the compiler sees the word `class`, it understands that the word after `class` is the name for the class.

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Java Reserved words

abstract	assert	boolean	break	long	native	new	package
byte	case	catch	char	private	protected	public	return
class	const	continue	default	short	static	strictfp	super
do	double	else	enum	switch	synchronized	this	throw
extends	final	finally	float	throws	transient	try	void
for	goto	if	implements	import	instanceof	int	interface
volatile	while						

Exercise:

Do you think that java reserved words and java keywords are different? If so, explain with examples.

Blocks

- A block is a group of statements or components of a program surrounded by a pair of braces (`{...}`).

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

The diagram illustrates the nesting of blocks in the provided Java code. A blue line connects the opening brace of the `public class Welcome` to its closing brace, with a box labeled "Class block" next to it. A green line connects the opening brace of the `public static void main` method to its closing brace, with a box labeled "Main block" next to it.

Appropriate Comments and Comment Styles

- Include a summary or a brief description at the beginning of the program to explain what the program does, its key features, your name, class section, date, and any unique techniques it uses.
- Comment styles in java:
 - ✓ Single line comments (begin with `//`)
 - ✓ Block comments (begin with `/*` and end with `*/`)
- Java also supports comments of a special type, referred to as **javadoc** comments.
 - ☞ javadoc comments begin with `/**` and end with `*/`.

Proper Indentation and Spacing

- Indentation is used to illustrate the structural relationships between a program's components or statements.
- A consistent indentation style makes programs clear and easy to read, debug, and maintain.
- ☞ Indentation
 - Indent two spaces.
- ☞ Spacing
 - Use blank line to separate segments of the code.
 - A single space should be added on both sides of a binary operator.

Block Styles

- There are two popular styles, next-line style and end-of-line style.
- If you want to minimize spaces, use end-of-line style.

**Next-line
style**

```
public class Test
{
    public static void main(String[] args)
    {
        System.out.println("Block Styles");
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Block Styles");
    }
}
```

**End-of-line
style**

Output Statements in Java

- Common java output statements:
 - ✓ `System.out.println( );`
 - ✓ `System.out.print( );`
 - ✓ `System.out.printf( );`
- The **System.out** object is known as the standard output object. It allows Java applications to display information in the command window from which it executes.
- Method **System.out.println** instructs the computer to display (or print) a line of text in the command window.

Print() and println() methods

For example:

System.out.println ("Hello World!"); // prints Hello World!

- You can print the value of a variable using the same commands we used to print Strings.

Example:

```
class Hello {  
    public static void main (String[] args) {  
        String s; // declares variable  
        s = "Hello, again!"; // assigns the value Hello, again to s  
        System.out.println (s); // prints the value of s  
    }  
}
```

Print() and println() methods...

- We can use the concatenation operator (+) to join two or more Strings and print them.

For example, Look at the following code:

```
int a=1;  
String s = "World";  
System.out.println(a + " " + "Hello " + s + "!");
```

The output would be:

1 Hello World!

- When two integers are operated with a + sign, the + acts as an addition operator and not a concatenation operator.

Print() and println() methods...

- Consider the following example:

```
int a = 3;  
int b = 4;  
System.out.println( a + b );  
System.out.println( "3" + "4" );  
System.out.println( " " + a + b );  
System.out.println( 3 + 4 + a + " " + b + a );  
System.out.println( "Result: " + a + b );  
System.out.println( "Result: " + ( a + b ) );
```

The output would be

```
7  
34  
34  
10 43  
Result: 34  
Result: 7
```

Print() and println() methods...

- Whenever a char variable is passed as an argument to the **print()** or **println()** methods, the graphic representation is printed and not the numeric value.

Example:

```
char a=65;  
char b='A';  
System.out.println(a);  
System.out.println(b);
```

The output would be

A

A

Print() and println() methods...

- If we want the numeric representation to be printed, we need to cast the variable into one of the four integer types.

```
System.out.println((int)a);
```

The output would be

65

- When an integer is added to a char, the char is implicitly casted to an integer.

```
System.out.println(b + 1);
```

The output would be

66

printf() method

- The **System.out.printf** method (f means “formatted”) displays formatted data.
- Formatting here is related to how data is to be displayed:
 - ✓ the number of decimal digits to be displayed,
 - ✓ the alignment of the String, etc.
- The signature of **printf ()** method is:
System.out.printf(“format string”, argument lists);
- A format String consists of **fixed text** and **format specifiers(placeholders)**.

Common Format Specifiers

Format Specifiers	Description
<code>%d</code>	Displays a decimal (base 10) integer
<code>%f</code>	Displays a floating point value in decimal format
<code>%e</code> or <code>%E</code>	Displays a floating point number in exponential notation
<code>%c</code> or <code>%C</code>	Displays characters
<code>%s</code> or <code>%S</code>	Displays Strings
<code>%b</code> or <code>%B</code>	Displays boolean values
<code>%%</code>	Displays a % sign
<code>%n</code>	Displays a newline

printf() method...

- The following code fragment shows the usage of these specifiers.

```
int day = 9;  
String month = "March";  
int year = 1983;  
char grade='A';  
int marks=99;  
System.out.printf("Born on %dth %s, %d%n", day, month, year);  
System.out.printf("Scored %d%% marks and grade %c%n ",  
marks, grade);
```

The output would be:

```
Born on 9th March, 1983  
Scored 99% marks and grade A
```

printf() method...

- To justify values we use integers.
- To left justify the data, we place a negative sign before the number.
- To right justify the data, we simply place the number.

```
String name1="Abebe";  
String name2="Kebede";  
int marks1=100;  
int marks2=99;  
System.out.printf("%-10s - %4d%n",name1,marks1);  
System.out.printf("%-10s - %4d%n",name2,marks2);
```

The output would be

```
Abebe      - 100  
Kebede     -  99
```

printf() method...

- Precision is specified by placing a dot (.) followed by a number indicating the required precision.

System.out.printf("%.3f", 34.789456);

The output would be

34.789

- Zero(0) flag in a printf format specification (e.g., "%02d") is used to display leading zeros for a value that doesn't use all the character positions in the specified field width.

Example: **System.out.printf("%02d%n%02d%n", 9, 12);**

The output would be

09

12

Escape Sequences

- Escape sequences are a combination of a backslash character '\' followed by one or more letters.
- The backslash(\) is an escape character.
- When a backslash appears in a string, Java combines it with the next character to form an escape sequence.

Conti...

- Java language supports few special escape sequences for String and char literals.

Escape sequence	Description
<code>\n</code>	Newline
<code>\t</code>	Horizontal tab
<code>\r</code>	Carriage return – ignores everything left of it
<code>\s</code>	Space
<code>\”</code>	Double quote
<code>\’</code>	Single quote
<code>\\</code>	Backslash

Conti...

Example:

```
String str="Hello World!";
```

```
System.out.println (str + "\nA new line"+"\t" +  
"Tab"+"\n" + "A new line");
```

The output would be

Hello World!

A new line Tab

A new line

Conti...

- Escape sequences should be placed in double quotes like strings, but if you put in a single quote logic error will occur.
- Look at the following code where a logical error exists.
- The intended output is the numbers 3,4 and 7 on different lines but the actual output is something different. Why?

```
int a=3, b = 4, c = 7;  
System.out.println(a + '\n' + b + '\n' + c);
```

The output is

34

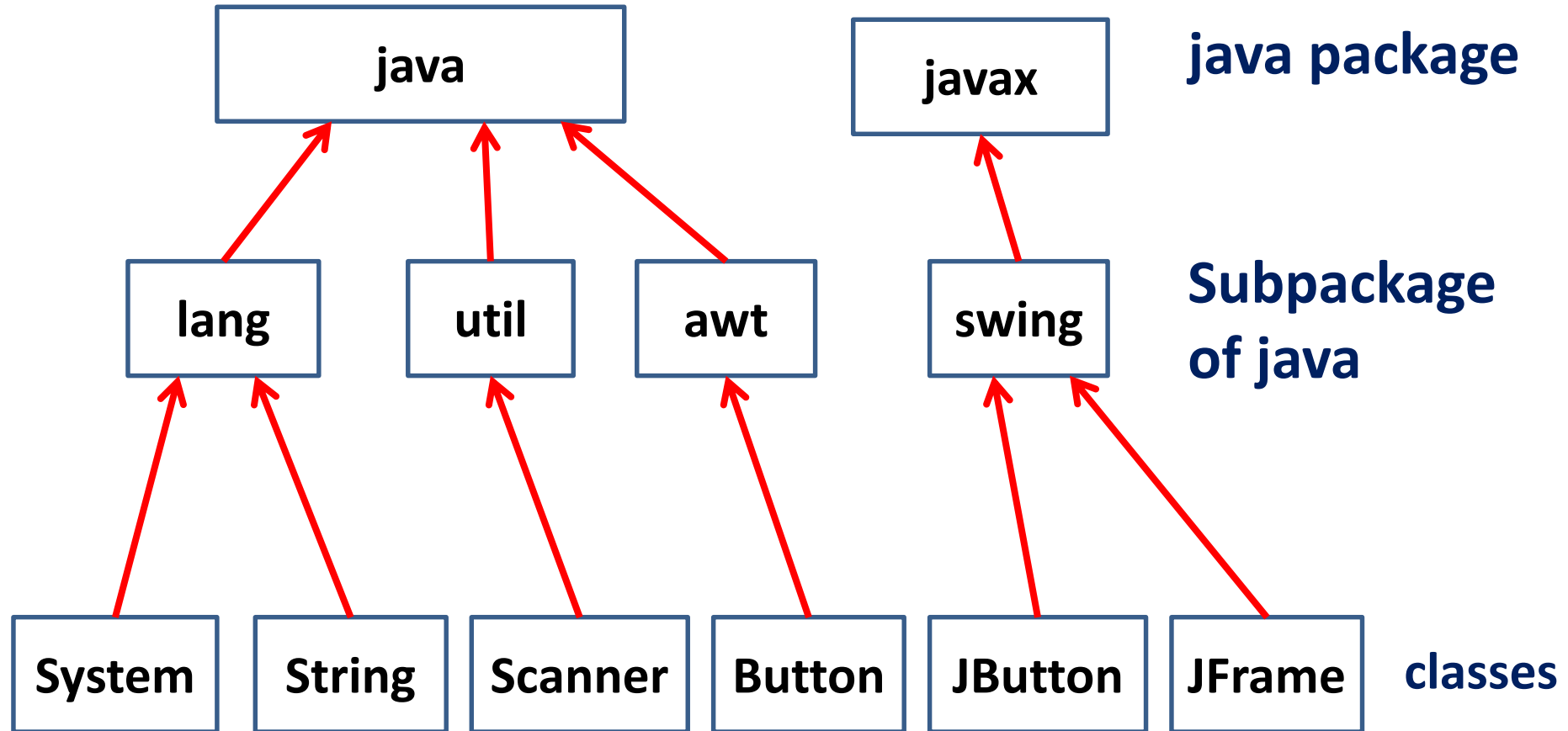
Packages in Java

- A **java package** is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form, built-in package and user-defined package.
- There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Advantages of Java Package:

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming conflicts.

Sample built-in packages



Simple example of java package

- The **package keyword** is used to create a package in java.

//save as Simple.java

```
package mypack; // should be the first statement
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

How to access package from another package?

- There are three ways to access the package from outside the package.
 - `import package.*;`
 - `import package.classname;`
 - fully qualified name.

Using packagename.*

- If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.
- The import keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

//save by A.java

```
package pack;
public class A{
    public void msg(){
        System.out.println("Hello");
    }
}
```

//save by B.java

```
package mypack;
import pack.*;
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

Using package.name.classname

- If you import package.classname then only declared class of this package will be accessible.

Example of package by import package.classname

//save by A.java

```
package pack;  
public class A{  
    public void msg() {  
        System.out.println("Hello");  
    }  
}
```

//save by B.java

```
package mypack;  
import pack.A;  
class B{  
    public static void main(String args[]){  
        A obj = new A();  
        obj.msg();  
    }  
}
```

Using fully qualified name

- If you use fully qualified name then only declared class of this package will be accessible.
- It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example of package by import fully qualified name

//save by A.java

```
package pack;
public class A{
    public void msg() {
        System.out.println("Hello");
    }
}
```

//save by B.java

```
package mypack;
class B{
    public static void main(String args[]){
        //using fully qualified name
        pack.A obj = new pack.A();
        obj.msg();
    }
}
```

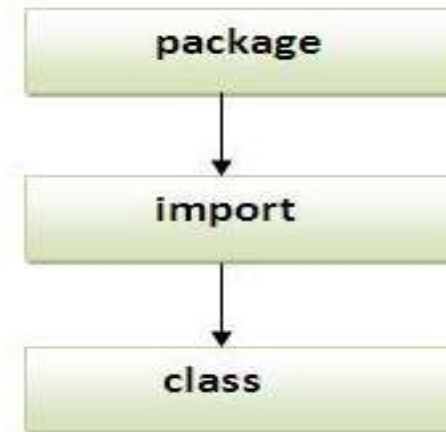
Subpackage

- If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages.
- Hence, you need to import the subpackage as well.

Example of Subpackage

```
/*The standard of defining package is  
domain.company.package */  
package com.javatpoint.core;  
class Simple{  
    public static void main(String args[]){  
        System.out.println("Hello subpackage");  
    }  
}
```

Note: Sequence of the program must be package then import then class.



How to put two public classes in a package?

- If you want to put two public classes in a package, have two java source files containing one public class, but keep the package name same.

For example:

//save as A.java

```
package javatpoint;  
public class A{  
}
```

//save as B.java

```
package javatpoint;  
public class B{  
}
```

Reading Input from the Console

- The **System** class which we have used contains three basic pre-defined streams:
 - ✓ Standard output stream (System.**out**)
 - ✓ Standard input stream (System.**in**)
 - ✓ Standard error stream (System.**err**)
- 'System' is a predefined class and '**in**' is a static variable in that class which holds a reference to an input stream object.
- We now use this **in** object to create a Scanner object.

Reading Input from the Console...

- You can use the **Scanner** class to create an object to read input from **System.in**, as follows:

Scanner input = new Scanner(System.in);

- A Scanner constructor requires an input stream object as an argument.
- Therefore we pass the **in** object to the Scanner constructor and create a Scanner object reference named **input** as above.
- The Scanner class has several methods which are used to take different types of inputs.
- They are listed in the table below:

Cont'd...

Method	Description
nextByte()	Accepts a byte
nextShort()	Accepts a short integer
nextInt()	Accepts an integer
nextLong()	Accepts a long integer
nextFloat()	Accepts a float
nextDouble()	Accepts a double
nextBoolean()	Accepts a boolean
next()	Accepts a single word
nextLine()	Accepts a line of String

Cont'd...

- Suppose, we want to accept an integer and store it in the variable firstNumber, we write the following statement:

```
Scanner sc = new Scanner(System.in);
```

```
int firstNumber = sc.nextInt();
```

- The following code shows the complete program which accepts two numbers from the user and adds them and displays the result.

Cont'd...

```
import java.util.Scanner;

public class Addition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int firstNumber = sc.nextInt();
        System.out.print("Enter second number: ");
        int secondNumber = sc.nextInt();
        int sum = firstNumber + secondNumber;
        System.out.println("\nThe sum of addition is " + sum);
    }
}
```

Exercise

- Write a java program that can prompt users to input a string as student name and three integers from the keyboard as the marks of a student in three subjects and display the student's name and average as follow.

Enter your name: Mohammed

Enter marks in three subjects: 100 98 99

Your name is: Mohammed

Your average is: 99.0